TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ THI GIỮA HỌC KỲ VÀ BÀI LÀM

Tên học phần: Toán ứng dụng CNTT
Mã học phần: Hình thức thi: *Tự luận*
Đề số: **04** Thời gian làm bài: 90 phút *(không kể thời gian chép/phát đề)*
Được sử dụng tài liệu khi làm bài.

Họ tên: Huỳnh Mỹ Ngọc **Lớp**: 24T_DT4 **MSSV**: 102240207

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MSTeam

Câu 1 (***3 điểm***): Cho số nguyên dương N= 40043. Viết chương trình bằng C/C++ có sử dụng hàm thực hiện:

- Phân tích thừa số nguyên tố đối với số N
- Tìm số M là ước số nguyên tố lớn nhất của N
- Tìm số lượng các số nguyên tố nằm trong khoảng từ 1000 đến M, liệt kê và tính tổng của chúng.
- Tìm số lượng các số hoàn hảo nhỏ hơn M, liệt kê và tính tổng của chúng.

Trả lời: Dán code vào bên dưới:

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Kiểm tra số nguyên tố
bool isPrime(long long n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;

    for (long long i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}

// Phân tích thừa số nguyên tố của N
void primeFactorization(long long n) {
    cout << "Phan tich thua so nguyen to cua N: ";

    bool first = true;

    while (n % 2 == 0) {
        if (!first) cout << " * ";
        cout << 2;
        first = false;
        n /= 2;
    }
```

```
    for (long long i = 3; i * i <= n; i += 2) {
        while (n % i == 0) {
            if (!first) cout << " * ";
            cout << i;
            first = false;
            n /= i;
        }
    }

    if (n > 1) {
        if (!first) cout << " * ";
        cout << n;
    }

    cout << endl;
}

// Tìm ước số nguyên tố lớn nhất của N
long long largestPrimeFactor(long long n) {
    long long maxPrime = -1;

    while (n % 2 == 0) {
        maxPrime = 2;
        n /= 2;
    }

    for (long long i = 3; i * i <= n; i += 2) {
        while (n % i == 0) {
            maxPrime = i;
            n /= i;
        }
    }

    if (n > 2) maxPrime = n;

    return maxPrime;
}

// Sàng Eratosthenes tìm các số nguyên tố trong khoảng (L ; R)
vector<int> sievePrimesInRange(int L, int R) {
    vector<int> result;

    if (R < 2 || L > R) return result;

    vector<bool> prime(R + 1, true);
    prime[0] = false;
    prime[1] = false;

    for (int i = 2; i * i <= R; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= R; j += i) {
                prime[j] = false;
```

```
            }
        }
    }

    for (int i = L + 1; i < R; i++) {
        if (prime[i]) result.push_back(i);
    }

    return result;
}

// Sinh các số hoàn hảo nhỏ hơn M
vector<long long> perfectNumbersLessThanM(long long M) {
    vector<long long> perfects;

    for (int p = 2; ; p++) {
        long long mersenne = (1LL << p) - 1;   // 2^p - 1

        if (!isPrime(mersenne)) continue;

        long long perfect = (1LL << (p - 1)) * mersenne;

        if (perfect >= M) break;

        perfects.push_back(perfect);
    }

    return perfects;
}

int main() {
    long long N = 40043;

    cout << "N = " << N << endl;

    // 1. Phân tích thừa số nguyên tố của N
    primeFactorization(N);

    // 2. Tìm M là ước số nguyên tố lớn nhất của N
    long long M = largestPrimeFactor(N);
    cout << "Uoc so nguyen to lon nhat cua N la M = " << M << endl;

    // 3. Tìm các số nguyên tố trong khoảng từ 1000 đến M
    vector<int> primes = sievePrimesInRange(1000, (int)M);
    long long sumPrime = 0;

    cout << "Cac so nguyen to trong khoang tu 1000 den " << M << " la: ";
    for (int x : primes) {
        cout << x << " ";
        sumPrime += x;
    }
    cout << endl;
```

```
    cout << "So luong cac so nguyen to: " << primes.size() << endl;
    cout << "Tong cac so nguyen to: " << sumPrime << endl;

    // 4. Tìm các số hoàn hảo nhỏ hơn M
    vector<long long> perfects = perfectNumbersLessThanM(M);
    long long sumPerfect = 0;

    cout << "Cac so hoan hao nho hon " << M << " la: ";
    for (long long x : perfects) {
        cout << x << " ";
        sumPerfect += x;
    }
    cout << endl;
    cout << "So luong cac so hoan hao: " << perfects.size() << endl;
    cout << "Tong cac so hoan hao: " << sumPerfect << endl;

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
N = 40043
Phan tich thua so nguyen to cua N: 23 * 1741
Uoc so nguyen to lon nhat cua N la M = 1741
Cac so nguyen to trong khoang tu 1000 den 1741 la: 1009 1013 1019 1021 1031 1033 1039 1049 1051 1061 1063 1069 1087 1091 1093 1097 1103 1109 1117 1123 1129 1151 1153 1163 1171 1181 1187
 1193 1201 1213 1217 1223 1229 1231 1237 1249 1259 1277 1279 1283 1289 1291 1297 1301 1303 1307 1319 1321 1327 1361 1367 1373 1381 1399 1409 1423 1427 1429 1433 1439 1447 1451 1453 1459
 1471 1481 1483 1487 1489 1493 1499 1511 1523 1531 1543 1549 1553 1559 1567 1571 1579 1583 1597 1601 1607 1609 1613 1619 1621 1627 1637 1657 1663 1667 1669 1693 1697 1699 1709 1721 1723
 1733
So luong cac so nguyen to: 102
Tong cac so nguyen to: 139144
Cac so hoan hao nho hon 1741 la: 6 28 496
So luong cac so hoan hao: 3
Tong cac so hoan hao: 530
```

***Câu 2*: (2 điểm)** *Cho hệ phương trình đồng dư sau*

$$\begin{cases} x \equiv 1 \bmod 3 \\ x \equiv 2 \bmod 5 \\ x \equiv 4 \bmod 7 \\ x \equiv 6 \bmod 11 \\ x \equiv 9 \bmod 17 \end{cases}$$

- Viết chương trình C/C++ có sử dụng hàm giải hệ phương trình đồng dư trên.

Trả lời: Dán code vào bên dưới:

```
#include <iostream>
using namespace std;

long long extendedGCD(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extendedGCD(b, a % b, x1, y1);
    x = y1;
```

```
    y = x1 - y1 * (a / b);
    return d;
}

long long modInverse(long long a, long long m) {
    long long x, y;
    long long g = extendedGCD(a, m, x, y);
    if (g != 1) return -1;
    return (x % m + m) % m;
}

// Chinese Remainder Theorem
long long CRT(int n, long long a[], long long m[]) {
    long long M = 1;
    for (int i = 0; i < n; i++) {
        M *= m[i];
    }

    long long result = 0;

    for (int i = 0; i < n; i++) {
        long long Mi = M / m[i];
        long long yi = modInverse(Mi, m[i]);

        if (yi == -1) {
            cout << "Khong ton tai nghich dao, he vo nghiem!" << endl;
            return -1;
        }

        result = (result + a[i] * Mi * yi) % M;
    }

    return (result + M) % M;
}

int main() {
    int n;
    cout << "Nhap so phuong trinh: ";
    cin >> n;

    long long a[100], m[100];

    cout << "Nhap dang: x ≡ a (mod m)" << endl;

    for (int i = 0; i < n; i++) {
        cout << "Nhap a[" << i+1 << "] va m[" << i+1 << "]: ";
        cin >> a[i] >> m[i];
    }

    long long x = CRT(n, a, m);

    if (x != -1) {
```

```
        long long M = 1;
        for (int i = 0; i < n; i++) M *= m[i];

        cout << "Nghiem x = " << x << endl;
        cout << "Dang tong quat: x = " << x << " + k*" << M << endl;
    }

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Nhap so phuong trinh: 5
Nhap dang: x ≡ a (mod m)
Nhap a[1] va m[1]: 1 3
Nhap a[2] va m[2]: 2 5
Nhap a[3] va m[3]: 4 7
Nhap a[4] va m[4]: 6 11
Nhap a[5] va m[5]: 9 17
Nghiem x = 4582
Dang tong quat: x = 4582 + k*19635
PS D:\UDCNTT\GK\LY\Lecture1>
```

Câu 3: (**5** ***điểm***) Cho ma trận

$$A=\begin{bmatrix} 9 & 2 & 1 \\ 2 & 11 & 3 \\ 1 & 3 & 10 \end{bmatrix}.$$

Viết chương trình bằng c/c++ có sử dụng hàm thực hiện phân rã ma trận A (có hàm kiểm tra điều kiện phân rã).

a) Phân rã **Cholesky** LDL^T ma trận A

Trả lời: Dán code vào bên dưới (bao gồm điều kiện của ma trận A nếu có):

```
#include <iostream>
#include <iomanip>
#include <cmath>

using namespace std;

const int MAX = 10;
const double EPS = 1e-5;

void inputMatrix(double A[MAX][MAX], int n, int m) {
    cout << "Nhap ma tran A:\n";
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
            cout << "A[" << i << "][" << j << "] = ";
            cin >> A[i][j];
        }
    }
}

bool isSquare(int n, int m) {
    return n == m;
}

bool isSymmetric(double A[MAX][MAX], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (fabs(A[i][j] - A[j][i]) > EPS)
                return false;
        }
    }
    return true;
}

bool LDLT_Decomposition(double A[MAX][MAX], double L[MAX][MAX], double D[MAX], int n)
{
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            L[i][j] = 0;
        }
    }

    for (int i = 0; i < n; i++) {
        L[i][i] = 1.0;
    }

    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int k = 0; k < i; k++) {
            sum += L[i][k] * L[i][k] * D[k];
        }
        D[i] = A[i][i] - sum;

        if (fabs(D[i]) < EPS) {
            return false;
        }

        for (int j = i + 1; j < n; j++) {
            double s = 0.0;
            for (int k = 0; k < i; k++) {
                s += L[j][k] * L[i][k] * D[k];
            }
            L[j][i] = (A[j][i] - s) / D[i];
        }
    }
```

```
    return true;
}

void printMatrixL(double L[MAX][MAX], int n) {
    cout << "Ma tran L:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << setw(10) << fixed << setprecision(5) << L[i][j];
        }
        cout << '\n';
    }
}

void printMatrixD(double D[MAX], int n) {
    cout << "Ma tran D:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) cout << setw(10) << fixed << setprecision(5) << D[i];
            else        cout << setw(10) << fixed << setprecision(5) << 0.0;
        }
        cout << '\n';
    }
}

void printTransposeL(double L[MAX][MAX], int n) {
    cout << "Ma tran L^T:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << setw(10) << fixed << setprecision(5) << L[j][i];
        }
        cout << '\n';
    }
}

int main() {
    double A[MAX][MAX], L[MAX][MAX], D[MAX];
    int n, m;

    cout << "Nhap so dong n = ";
    cin >> n;
    cout << "Nhap so cot m = ";
    cin >> m;

    if (!isSquare(n, m)) {
        cout << "Ma tran khong vuong, khong the phan ra LDL^T.\n";
        return 0;
    }

    inputMatrix(A, n, m);

    if (!isSymmetric(A, n)) {
```

```
        cout << "Ma tran khong doi xung, khong the phan ra LDL^T.\n";
        return 0;
    }

    if (!LDLT_Decomposition(A, L, D, n)) {
        cout << "Khong phan ra duoc LDL^T (gap pivot bang 0 hoac rat gan 0).\n";
        return 0;
    }

    cout << "\nPhan ra LDL^T thanh cong:\n";
    printMatrixL(L, n);
    cout << '\n';
    printMatrixD(D, n);
    cout << '\n';
    printTransposeL(L, n);

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới (sai số $\varepsilon = 10^{-5}$):

```
Nhap so dong n = 3
Nhap so cot m = 3
Nhap ma tran A:
A[0][0] = 9
A[0][1] = 2
A[0][2] = 1
A[1][0] = 2
A[1][1] = 11
A[1][2] = 3
A[2][0] = 1
A[2][1] = 3
A[2][2] = 10

Phan ra LDL^T thanh cong:
Ma tran L:
   1.00000   0.00000   0.00000
   0.22222   1.00000   0.00000
   0.11111   0.26316   1.00000

Ma tran D:
   9.00000   0.00000   0.00000
   0.00000  10.55556   0.00000
   0.00000   0.00000   9.15789

Ma tran L^T:
   1.00000   0.22222   0.11111
   0.00000   1.00000   0.26316
   0.00000   0.00000   1.00000
PS D:\UDCNTT\GK\LY\Lecture1>
```

b) Phân rã **eigendecomposition** ma trận A

Trả lời: Dán code vào bên dưới (bao gồm điều kiện của ma trận A nếu có):

```cpp
#include <iostream>
#include <vector>
#include <Eigen/Dense>

using namespace std;
using namespace Eigen;

MatrixXd nhapMatran(int n) {
    MatrixXd a(n, n);
    cout << "Nhập ma trận " << n << "x" << n << ":\n";
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
            cout << "Nhập phần tử ở vị trí (" << i + 1 << ", " << j + 1 << "): ";
            cin >> a(i, j);
        }
    }
    return a;
}

void xuatMatran(const MatrixXd& a, const string& name = "Ma trận") {
    cout << name << ":\n";
    for (int i = 0; i < a.rows(); ++i) {
        for (int j = 0; j < a.cols(); ++j) {
            cout << a(i, j) << " ";
        }
        cout << endl;
    }
    cout << endl;
}

// Hàm phân rã ma trận theo vector riêng và giá trị riêng
void phanRaEigen(const MatrixXd& a, VectorXd& eigenvalues, MatrixXd& eigenvectors) {
    EigenSolver<MatrixXd> solver(a);
    eigenvalues = solver.eigenvalues().real();
    eigenvectors = solver.eigenvectors().real();

    cout << "Giá trị riêng (Λ):\n";
    for (int i = 0; i < eigenvalues.size(); ++i) {
        cout << "λ" << i + 1 << " = " << eigenvalues(i) << endl;
    }
    cout << endl;

    cout << "Vector riêng (cột của Q):\n";
    for (int i = 0; i < eigenvectors.cols(); ++i) {
        cout << "v" << i + 1 << " = [";
        for (int j = 0; j < eigenvectors.rows(); ++j) {
            cout << eigenvectors(j, i) << " ";
        }
        cout << "]" << endl;
    }
    cout << endl;
}

int main() {
    try {
        int n;
        cout << "Nhập kích thước ma trận vuông n = ";
        cin >> n;
        if (n <= 0) {
            throw invalid_argument("Kích thước ma trận phải lớn hơn 0!");
        }

        MatrixXd a = nhapMatran(n);
```

```
        xuatMatran(a, "Ma trận đã nhập");

        // Phân rã eigen
        VectorXd eigenvalues(n);
        MatrixXd eigenvectors(n, n);
        phanRaEigen(a, eigenvalues, eigenvectors);

        MatrixXd Lambda = eigenvalues.asDiagonal();

        MatrixXd Q_inv = eigenvectors.inverse();

        // Hiển thị các thành phần phân rã
        xuatMatran(eigenvectors, "Ma trận vector riêng (Q)");
        xuatMatran(Lambda, "Ma trận giá trị riêng (Λ)");
        xuatMatran(Q_inv, "Ma trận nghịch đảo của vector riêng (Q⁻¹)");

        // Tái cấu tạo ma trận
        MatrixXd reconstruct = eigenvectors * Lambda * Q_inv;
        xuatMatran(reconstruct, "Ma trận tái cấu tạo (QΛQ⁻¹)");

        cout << "So sánh với ma trận ban đầu:\n";
        xuatMatran(a, "Ma trận ban đầu");
        xuatMatran(reconstruct, "Ma trận tái cấu tạo");

    } catch (const exception& e) {
        cout << "Lỗi: " << e.what() << endl;
    }

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới (sai số $\varepsilon = 10^{-5}$):

```
Nhập kích thước ma trận vuông n = 3
Nhập ma trận 3x3:
Nhập phần tử ở vị trí (1, 1): 9
Nhập phần tử ở vị trí (1, 2): 2
Nhập phần tử ở vị trí (1, 3): 1
Nhập phần tử ở vị trí (2, 1): 2
Nhập phần tử ở vị trí (2, 2): 11
Nhập phần tử ở vị trí (2, 3): 3
Nhập phần tử ở vị trí (3, 1): 1
Nhập phần tử ở vị trí (3, 2): 3
Nhập phần tử ở vị trí (3, 3): 10
Ma trận đã nhập:
9 2 1
2 11 3
1 3 10

Giá trị riêng (Λ):
λ1 = 14.4188
λ2 = 8.38677
λ3 = 7.1944

Vector riêng (cột của Q):
v1 = [0.374359 0.725619 0.57735 ]
v2 = [0.815583 0.0386051 -0.57735 ]
v3 = [0.441225 -0.687013 0.57735 ]

Ma trận vector riêng (Q):
0.374359 0.815583 0.441225
0.725619 0.0386051 -0.687013
0.57735 -0.57735 0.57735

Ma trận giá trị riêng (Λ):
14.4188 0 0
0 8.38677 0
0 0 7.1944
```

```
Ma trận nghịch đảo của vector riêng (Q⁻¹):
0.374359 0.725619 0.57735
0.815583 0.0386051 -0.57735
0.441225 -0.687013 0.57735

Ma trận tái cấu tạo (QΛQ⁻¹):
9 2 1
2 11 3
1 3 10

So sánh với ma trận ban đầu:
Ma trận ban đầu:
9 2 1
2 11 3
1 3 10

Ma trận tái cấu tạo:
9 2 1
2 11 3
1 3 10

PS D:\UDCNTT\GK\LY\Lecture1>
```